

PRACTICAL NO. 2

Introduction to Team Foundation Server (TFS) tool and account creation on Azure DevOps (the modern successor of TFS).

1. Objective

The objective of this practical is to:

- Understand the concept of Team Foundation Server (TFS) and its role in Application Lifecycle Management (ALM).
- Learn the difference between TFS (on-premises) and Azure DevOps Server / Azure DevOps Services (cloud).
- Create a personal account and a new project on the TFS/Azure DevOps platform.
- Explore the basic components: Boards, Repos, Pipelines, Test Plans, and Artifacts.
- Understand how TFS supports version control, work-item tracking, and automated testing in a software project.

2. Theory

2.1 What is Team Foundation Server (TFS)?

Team Foundation Server (TFS) is a Microsoft product that provides source code management (either through Team Foundation Version Control – TFVC, or Git), reporting, requirements management, project management, automated builds, testing, and release management capabilities. It is an Application Lifecycle Management (ALM) tool that covers the complete life cycle of a software product, right from planning to deployment and monitoring. Since 2019, Microsoft has rebranded the on-premises product as Azure DevOps Server, while the cloud-hosted version is called Azure DevOps Services — both use the same underlying technology as the original TFS.

2.2 Key Features of TFS

- Version Control – supports both centralized (TFVC) and distributed (Git) source control systems.
- Work Item Tracking – lets teams plan, track, and discuss work using boards, backlogs, and sprints (Agile/Scrum/CMMI process templates).
- Build Automation (CI) – automatically compiles and tests code whenever changes are checked in.
- Release Management (CD) – automates deployment of applications to various environments.
- Test Case Management – allows creation, execution, and tracking of manual and automated test cases.
- Reporting & Dashboards – provides real-time reports on project progress, bugs, and velocity.
- Integration – integrates with Visual Studio, Eclipse, and other third-party tools.

2.3 TFS Architecture (Basic Components)

- Application Tier – hosts the TFS web services and web portal.
- Data Tier – uses SQL Server to store all project data (work items, source code history, builds, etc.).
- Client Tier – tools like Visual Studio, Team Explorer, or a web browser used to interact with TFS.

2.4 TFS vs Azure DevOps vs GIT (Comparison)

Parameter	TFS / Azure DevOps	Git
-----------	--------------------	-----

Type	Centralized (TFVC) or Distributed (Git support builtin)	Purely Distributed Version Control System (DVCS)
-------------	---	--

Scope	Complete ALM suite: Boards, Repos, Pipelines, Test Plans, Artifacts	Only source code version control
Hosting	On-premises (TFS/Azure DevOps Server) or Cloud (Azure DevOps Services)	Local repository + remote host (GitHub, GitLab, Bitbucket, Azure Repos, etc.)
Ideal for	Enterprise teams needing end-to-end project and test management	Source code versioning and collaboration
Repository Model	Supports both centralized (TFVC) and distributed (Git) repositories	Fully distributed repositories
Offline Support	Limited in TFVC; full support with Git repositories	Full offline support
Branching	Available, but TFVC branching is more complex; Git branching is supported	Lightweight, fast, and easy branching & merging
Merging	Supported; easier with Git than TFVC	Fast and efficient merging

Performance	Better suited for large enterprise environments with integrated tools	Very fast for version control operations
Learning Curve	Moderate to High due to multiple integrated services	Moderate; mainly focuses on version control concepts
Project Management	Built-in Agile Boards, Backlogs, Sprints, Kanban, and Work Items	Not available (requires external tools)
CI/CD Support	Built-in Azure Pipelines for Continuous Integration and Deployment	Not built-in; uses external tools like GitHub Actions, Jenkins, GitLab CI/CD, etc.
Testing Support	Built-in Test Plans, Manual & Automated Testing	No built-in testing features
Security	Enterprise-grade security with Azure Active Directory integration and role-based access	Depends on the hosting platform (GitHub, GitLab, Bitbucket, etc.)
Scalability	Designed for large enterprise projects and organizations	Suitable for projects of all sizes, from personal to enterprise
Cost	Azure DevOps Services has free and paid plans; TFS/Azure DevOps Server requires licensing	Git is open-source and free; hosting services may have free and paid plans

Popular Tools	Azure DevOps Services, Azure DevOps Server (TFS)	Git, GitHub, GitLab, Bitbucket
----------------------	---	-----------------------------------

3. Software / Hardware Requirements

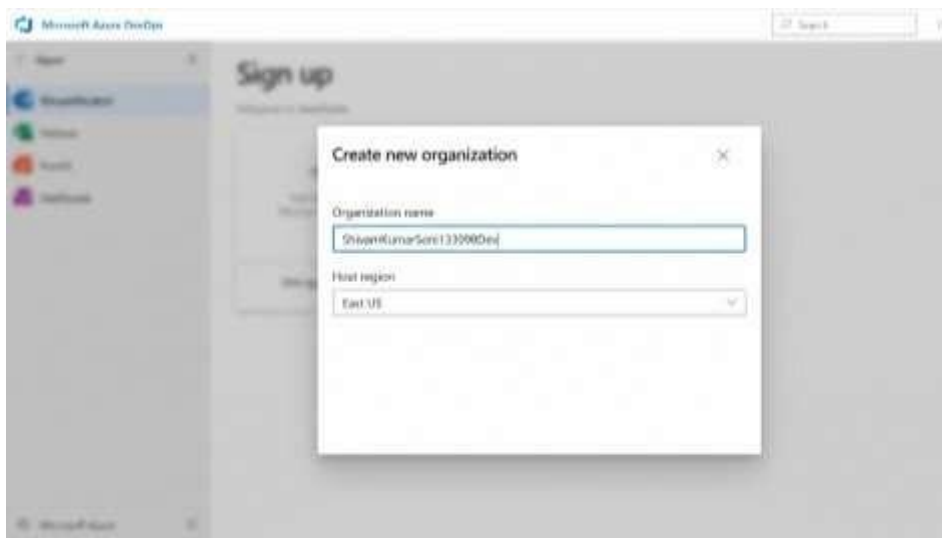
- A working internet connection
- Web browser (Google Chrome / Microsoft Edge)
- A valid Microsoft account (Outlook/Hotmail/Live ID) or organizational email ID
- Azure DevOps Services account (dev.azure.com) – free tier is sufficient for this practical

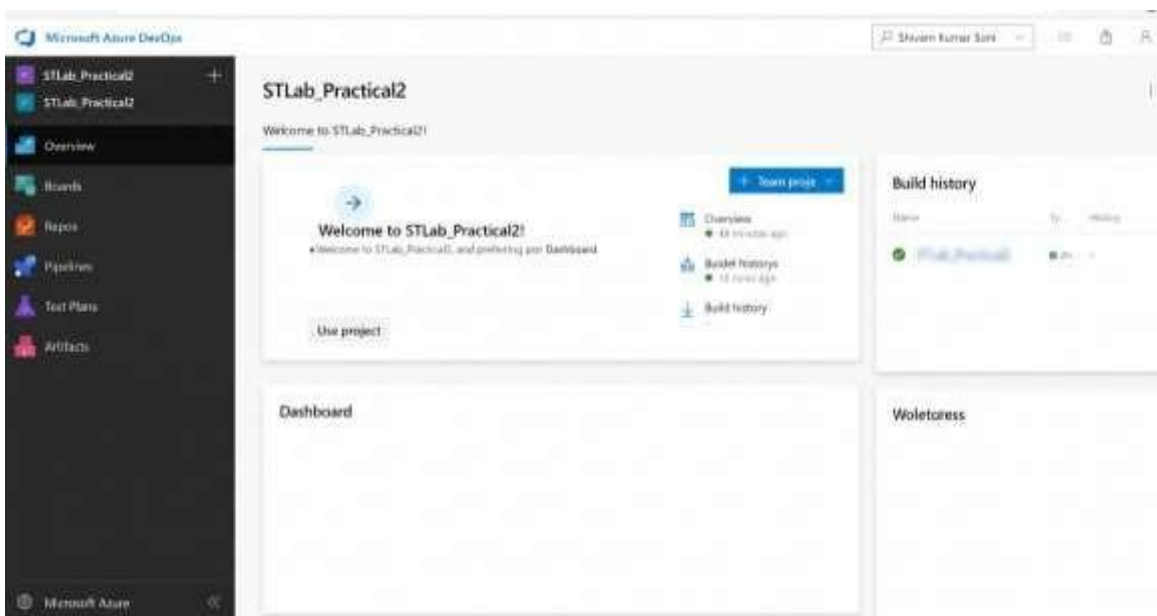
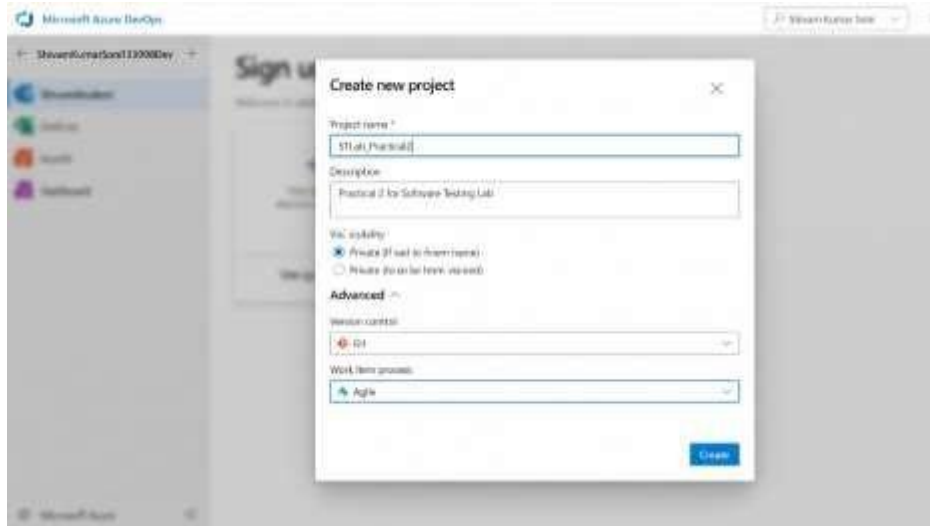
4. Procedure

1. Open a web browser and navigate to <https://dev.azure.com> (the cloudhosted version of TFS provided free by Microsoft).
2. Click on 'Start free' or 'Sign in' and log in using an existing Microsoft account, or create a new one by clicking 'Create one!'.
3. Once signed in, Azure DevOps will prompt you to create an organization. Enter a unique organization name (this acts as your TFS server namespace).
4. Select the region nearest to you for hosting the organization's data, then click 'Continue'.
5. After the organization is created, click on '+ New Project' to create a new project.
6. Enter the Project name (e.g., 'STLab_Practical2'), an optional description, and choose the Visibility (Private/Public).
7. Select the Version control type as Git (recommended) or Team Foundation Version Control (TFVC).

8. Choose the Work item process template — Agile, Scrum, or Basic — and click 'Create'.
9. Once the project is created, explore the left navigation panel consisting of: Overview, Boards, Repos, Pipelines, Test Plans, and Artifacts.
10. Under 'Boards', create a sample work item (e.g., a User Story or Task) to understand work item tracking.
11. Under 'Repos', note the clone URL of the Git repository created automatically for the project — this can be used with the Git client studied in Practical 1.
12. Invite a team member (optional) via Project Settings → Teams → Add member, to explore collaboration features.
13. Take a screenshot of the created project dashboard for submission along with this practical file.

5. Screenshot





6. Advantages of TFS

- Provides a single integrated platform for the entire software development life cycle.
- Improves team collaboration through shared boards and dashboards.
- Enables continuous integration and continuous deployment (CI/CD).

- Supports traceability — a code change can be linked back to a work item/requirement.
- Reduces dependency on multiple disconnected tools for version control, testing, and project tracking.

7. Disadvantages / Limitations

- Can have a steep learning curve for beginners compared to standalone Git.
- On-premises TFS requires significant hardware and SQL Server licensing costs.
- Heavier and more complex than lightweight version control tools for small projects.

8. Applications

- Used by enterprise software teams to manage large, complex projects.
- Used for Agile/Scrum project management with sprint planning and backlogs.
- Used to set up automated build and release pipelines (DevOps practice).
- Used for managing manual and automated test cases in the Test Plans module.

9. Conclusion

In this practical, the concept of Team Foundation Server (TFS) and its cloud counterpart Azure DevOps Services was studied. An account was successfully created on the Azure DevOps platform, and a new project was set up along with exploration of its core modules — Boards, Repos, Pipelines, and Test Plans. It was observed that TFS/Azure DevOps offers a complete Application Lifecycle Management solution, integrating version control, work item tracking, build automation, and test management into a single platform, which is extremely useful for managing real-world software testing and development projects.

